Microprocessor Interfacing and Programming(MIP)

Instructor: Dr. Muhammad Akmal

https://scholar.google.com/citations?hl=en&user=aqfWdG0AAAAJ&view_op=list_works&sortby=pubdate

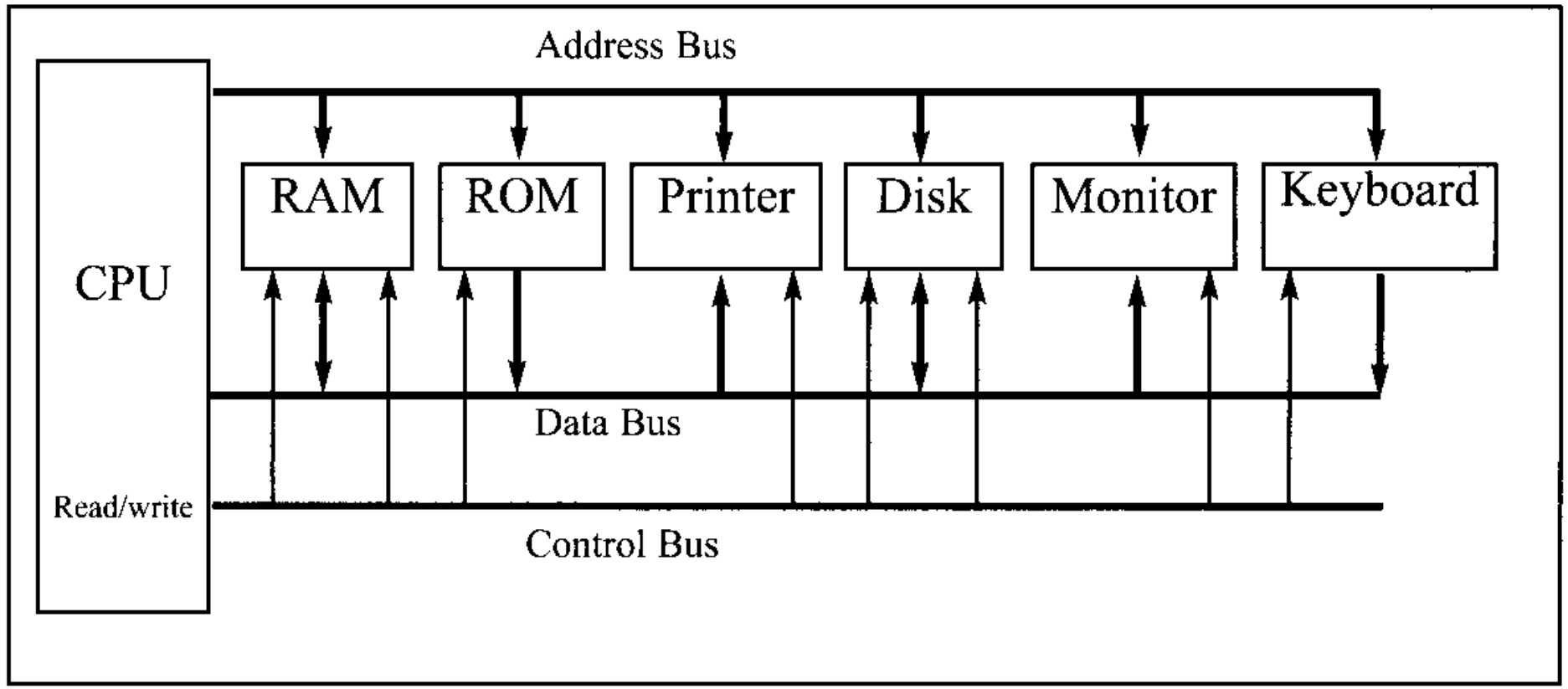


Figure 0-10. Internal Organization of a Computer

More about the address bus

Because the address bus is used to identify the devices and memory connected to the CPU, the more address buses available, the larger the number of devices that can be addressed. In other words, the number of address buses for a CPU determines the number of locations with which it can communicate. The number of locations is always equal to 2^x, where x is the number of address lines, regardless of the size of the data bus. For example, a CPU with 16 address lines can provide a total of 65,536 (2^{16}) or 64K of addressable memory. Each location can have a maximum of 1 byte of data. This is because all general-purpose microprocessor CPUs are what is called *byte addressable*.

An **address line** is a wire used by the CPU to specify **which memory location it wants to access**.

If a CPU has **16 address lines**, each line can carry either:

- 0, or
- 1

Therefore, the CPU can generate:

$$2^{16} = 65,536$$

different binary combinations.

Each combination represents a **unique memory address**.

So:

$$65,536 \text{ addresses} = 64K \text{ addresses}$$

$$64K = 64 \times 2^{10} = 2^6 \times 2^{10} = 2^{16} = 65536$$

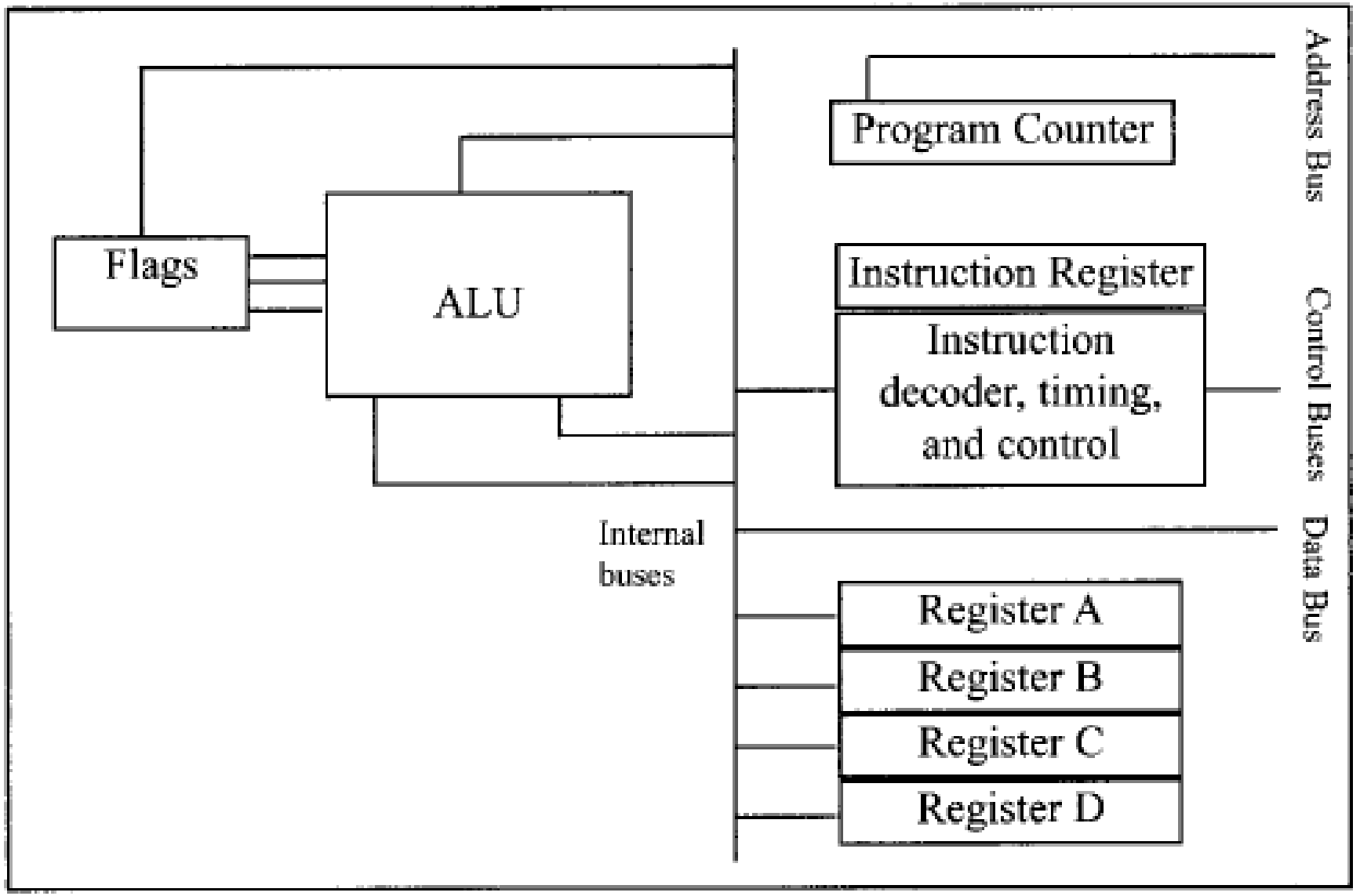


Figure 0-11. Internal Block Diagram of a CPU

Inside CPUs

A program stored in memory provides instructions to the CPU to perform an action. The action can simply be adding data such as payroll data or controlling a machine such as a robot. The function of the CPU is to fetch these instructions from memory and execute them. To perform the actions of fetch and execute, all CPUs are equipped with resources such as the following:

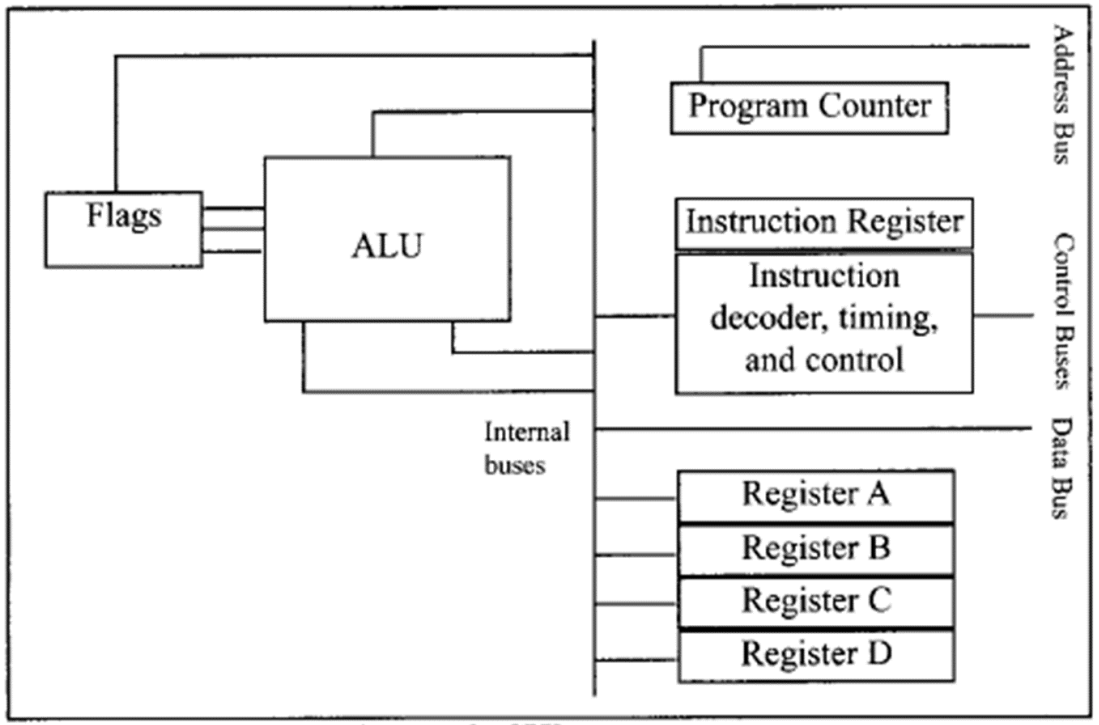


Figure 0-11. Internal Block Diagram of a CPU

1. Foremost among the resources at the disposal of the CPU are a number of *registers*. The CPU uses registers to store information temporarily. The information could be two values to be processed, or the address of the value needed to be fetched from memory. Registers inside the CPU can be 8-bit, 16-bit, 32-bit, or even 64-bit registers, depending on the CPU. In general, the more and bigger the registers, the better the CPU. The disadvantage of more and bigger registers is the increased cost of such a CPU.

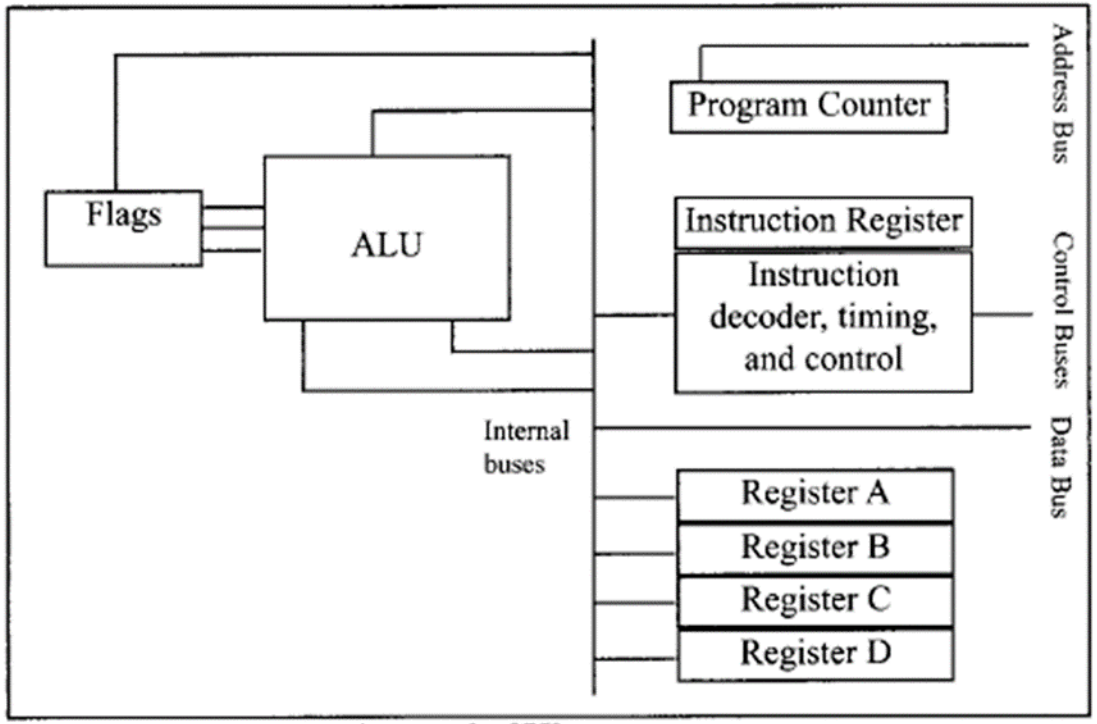


Figure 0-11. Internal Block Diagram of a CPU

2. The CPU also has what is called the *ALU* (arithmetic/logic unit). The ALU section of the CPU is responsible for performing arithmetic functions such as add, subtract, multiply, and divide, and logic functions such as AND, OR, and NOT.

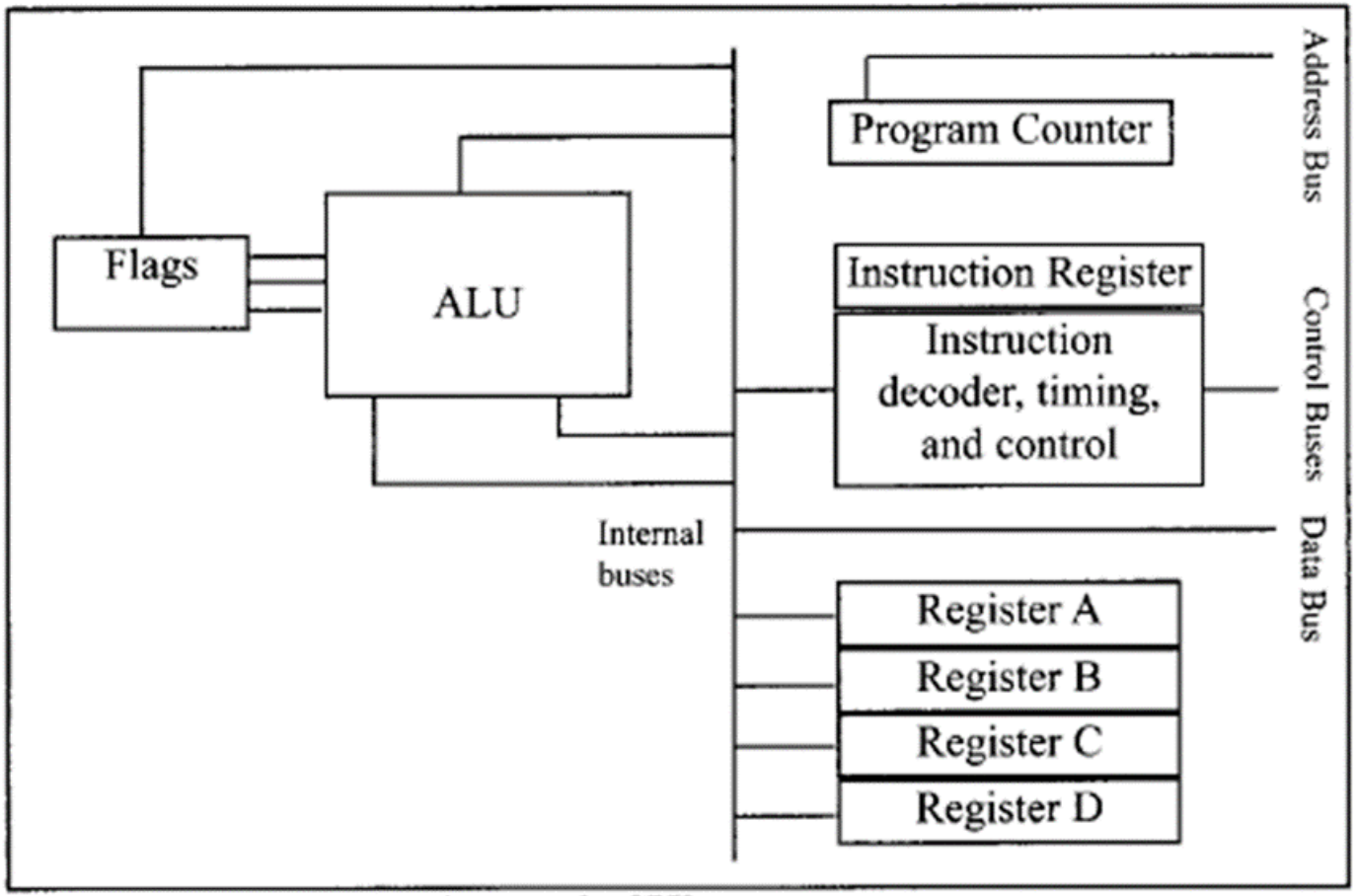


Figure 0-11. Internal Block Diagram of a CPU

3. Every CPU has what is called a *program counter*. The function of the program counter is to point to the address of the next instruction to be executed. As each instruction is executed, the program counter is incremented to point to the address of the next instruction to be executed. The contents of the program counter are placed on the address bus to find and fetch the desired instruction. In the IBM PC, the program counter is a register called IP, or the instruction pointer.

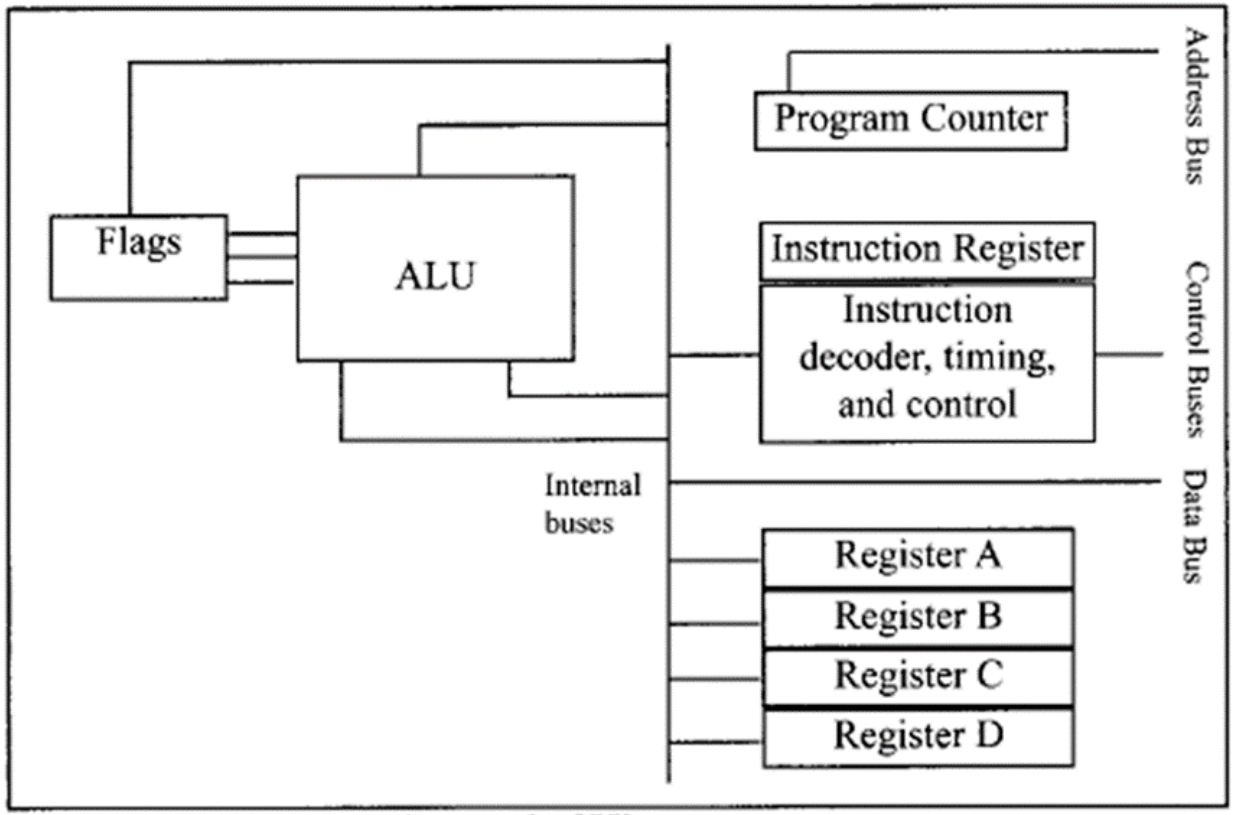


Figure 0-11. Internal Block Diagram of a CPU

4. The function of the *instruction decoder* is to interpret the instruction fetched into the CPU. One can think of the instruction decoder as a kind of dictionary, storing the meaning of each instruction and what steps the CPU should take upon receiving a given instruction. Just as a dictionary requires more pages the more words it defines, a CPU capable of understanding more instructions requires more transistors to design.

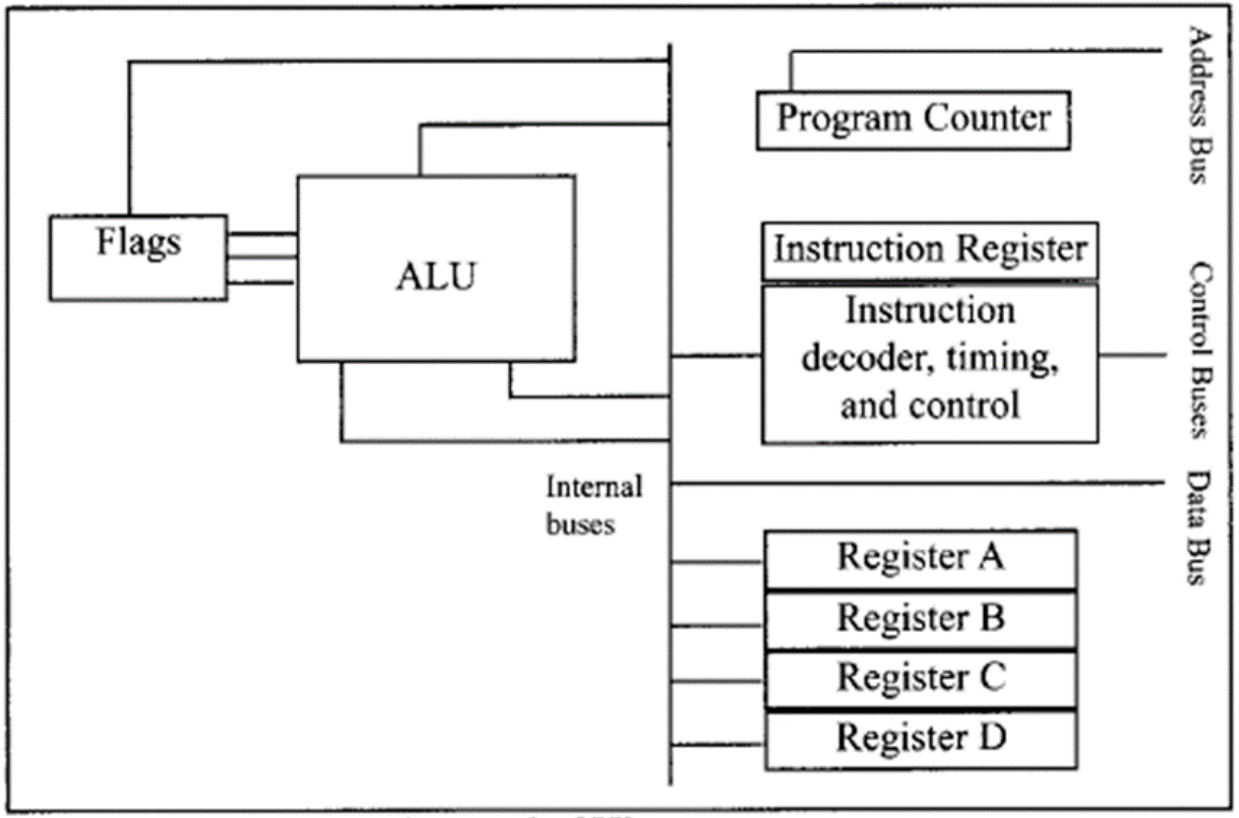


Figure 0-11. Internal Block Diagram of a CPU

Internal working of computers

FFFFH = 65535
From 0 to 65535 = 65536=10000H

To demonstrate some of the concepts discussed above, a step-by-step analysis of the process a CPU would go through to add three numbers is given next. Assume that an imaginary CPU has registers called A, B, C, and D. It has an 8-bit data bus and a 16-bit address bus. Therefore, the CPU can access memory from addresses 0000 to FFFFH (for a total of 10000H locations). The action to be performed by the CPU is to put hexadecimal value 21 into register A, and then add to register A values 42H and 12H. Assume that the code for the CPU to move a value to register A is 1011 0000 (B0H) and the code for adding a value to register A is 0000 0100 (04H). The necessary steps and code to perform them are as follows.

Action	*Code*	*Data*
Move value 21H into register A	B0H	21H
Add value 42H to register A	04H	42H
Add value 12H to register A	04H	12H

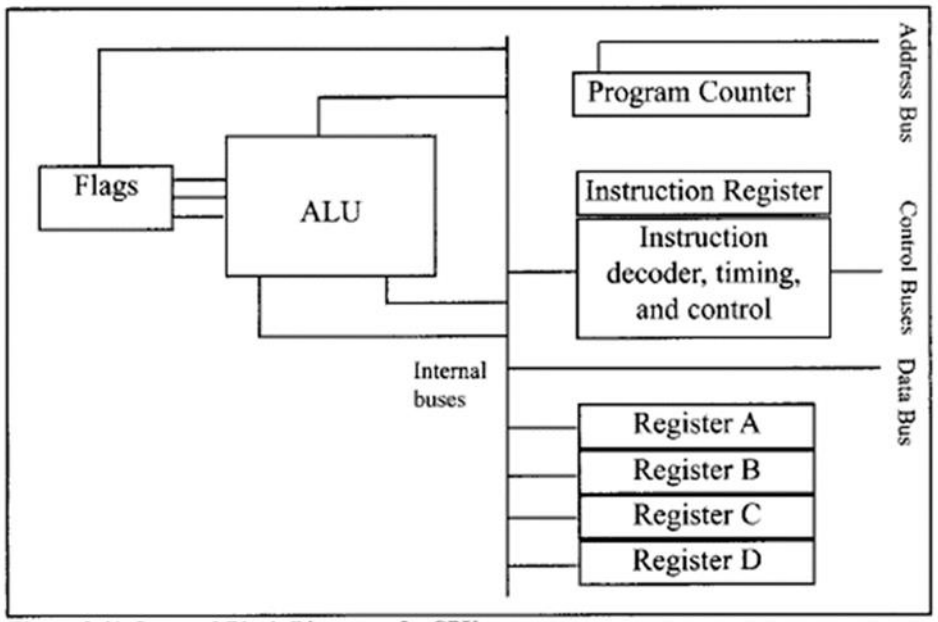


Figure 0-11. Internal Block Diagram of a CPU

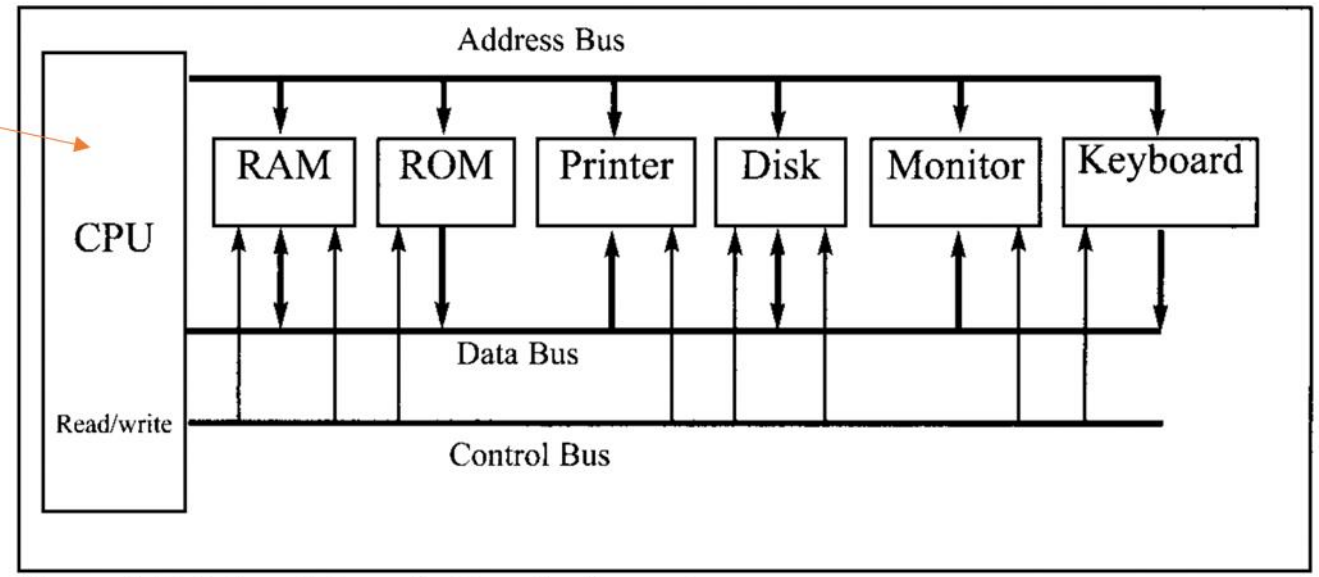


Figure 0-10. Internal Organization of a Computer

If the program to perform the actions listed above is stored in memory locations starting at 1400H, the following would represent the contents for each memory address location:

Memory address	***Contents of memory address***
1400	(B0)code for moving a value to register A
1401	(21)value to be moved
1402	(04)code for adding a value to register A
1403	(42)value to be added
1404	(04)code for adding a value to register A
1405	(12)value to be added
1406	(F4)code for halt

The actions performed by the CPU to run the program above would be as follows:

1. The CPU's program counter can have a value between 0000 and FFFFH. The program counter must be set to the value 1400H, indicating the address of the

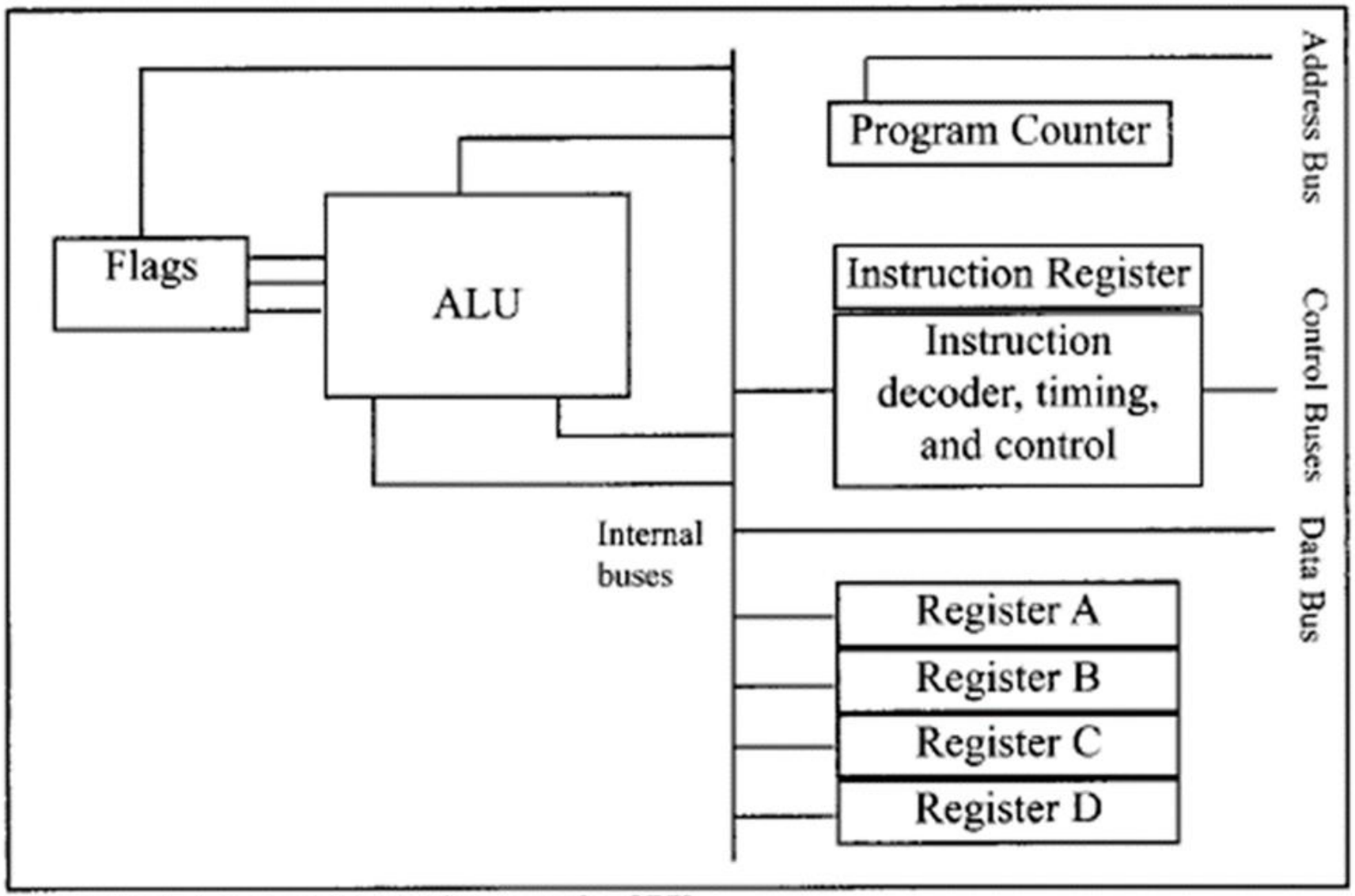


Figure 0-11. Internal Block Diagram of a CPU

first instruction code to be executed. After the program counter has been loaded with the address of the first instruction, the CPU is ready to execute.

2. The CPU puts 1400H on the address bus and sends it out. The memory circuitry finds the location while the CPU activates the READ signal, indicating to memory that it wants the byte at location 1400H. This causes the contents of memory location 1400H, which is B0, to be put on the data bus and brought into the CPU.

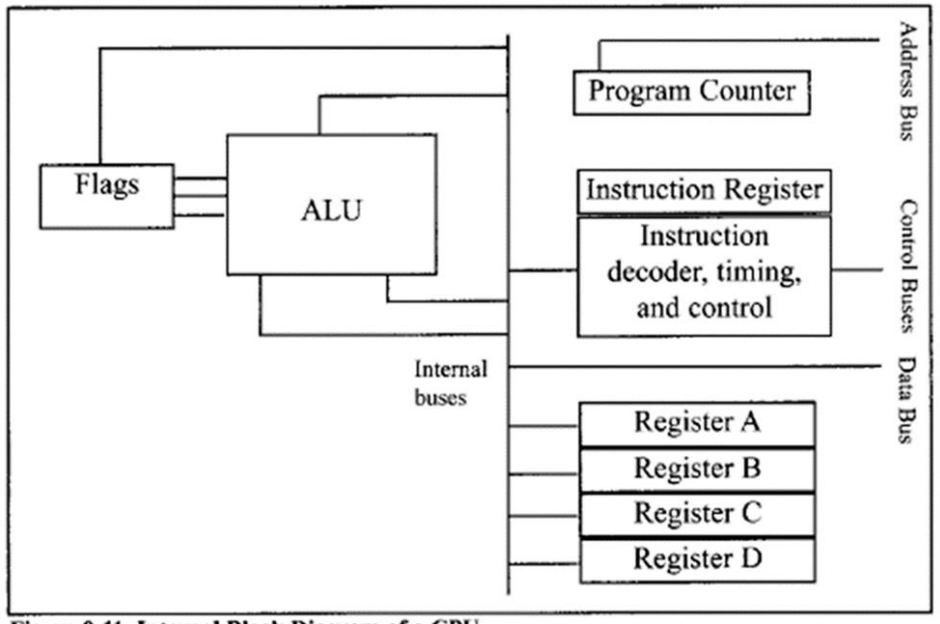


Figure 0-11. Internal Block Diagram of a CPU

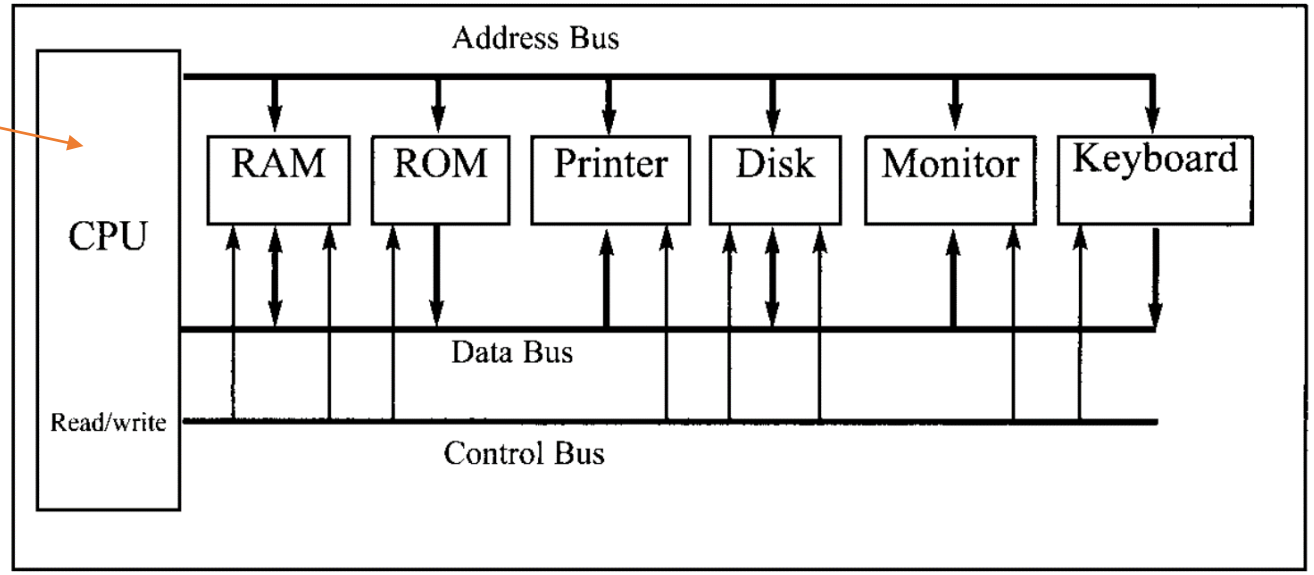


Figure 0-10. Internal Organization of a Computer

3. The CPU decodes the instruction B0 with the help of its instruction decoder dictionary. When it finds the definition for that instruction it knows it must bring into register A of the CPU the byte in the next memory location. Therefore, it commands its controller circuitry to do exactly that. When it brings in value 21H from memory location 1401, it makes sure that the doors of all registers are closed except register A. Therefore, when value 21H comes into the CPU it will go directly into register A. After completing one instruction, the program counter points to the address of the next instruction to be executed, which in this case is 1402H. Address 1402 is sent out on the address bus to fetch the next instruction.

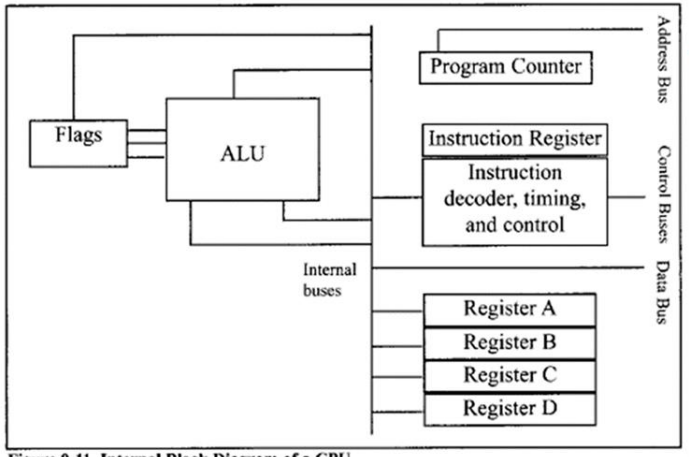


Figure 0-11. Internal Block Diagram of a CPU

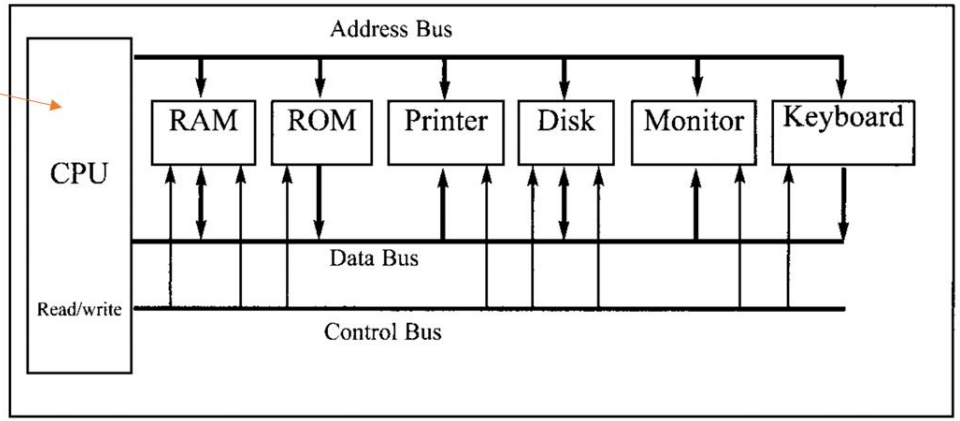


Figure 0-10. Internal Organization of a Computer

4. From memory location 1402H the CPU fetches code 04H. After decoding, the CPU knows that it must add the byte sitting at the next address (1403) to the contents of register A. After the CPU brings the value (in this case, 42H) into register A, it provides the contents of register A along with this value to the ALU to perform the addition. It then takes the result of the addition from the ALU's output and puts it in register A. Meanwhile the program counter becomes 1404, the address of the next instruction.

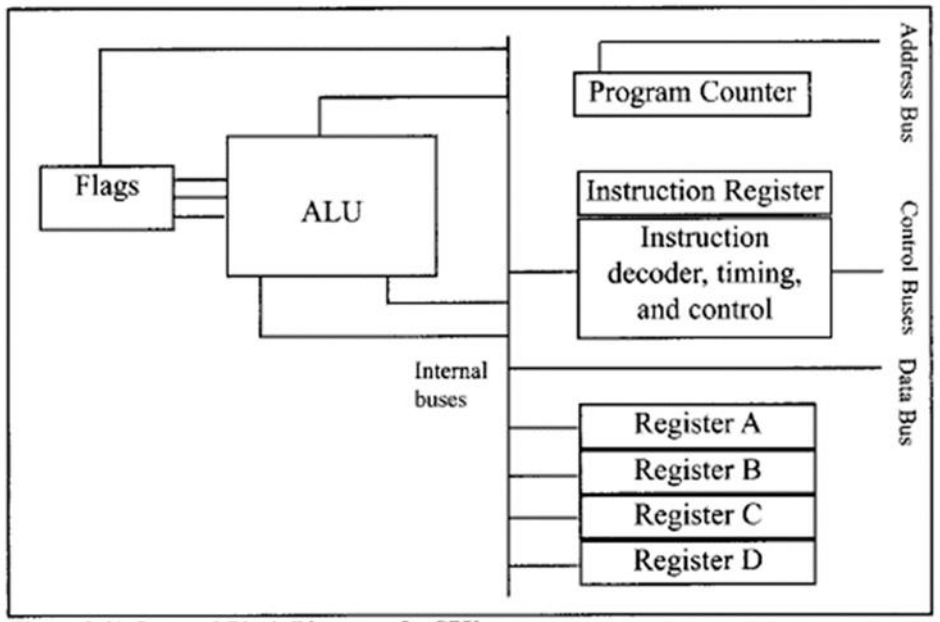


Figure 0-11. Internal Block Diagram of a CPU

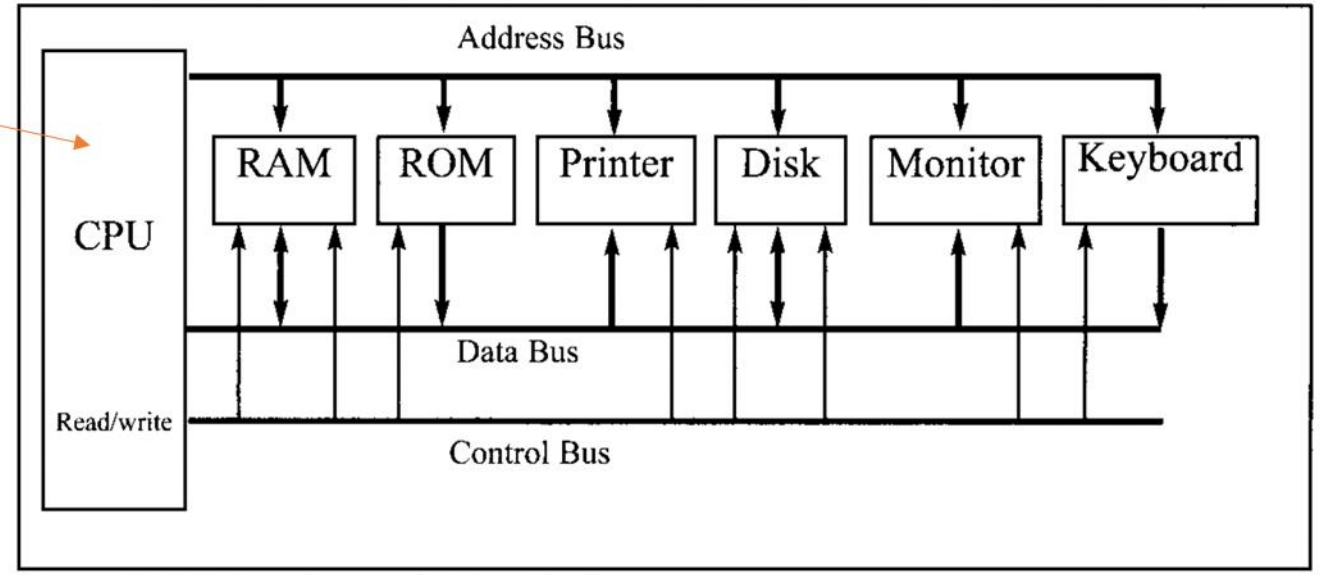


Figure 0-10. Internal Organization of a Computer

5. Address 1404H is put on the address bus and the code is fetched into the CPU, decoded, and executed. This code again is adding a value to register A. The program counter is updated to 1406H.
6. Finally, the contents of address 1406 are fetched in and executed. This HALT instruction tells the CPU to stop incrementing the program counter and asking for the next instruction. Without the HALT, the CPU would continue updating the program counter and fetching instructions.

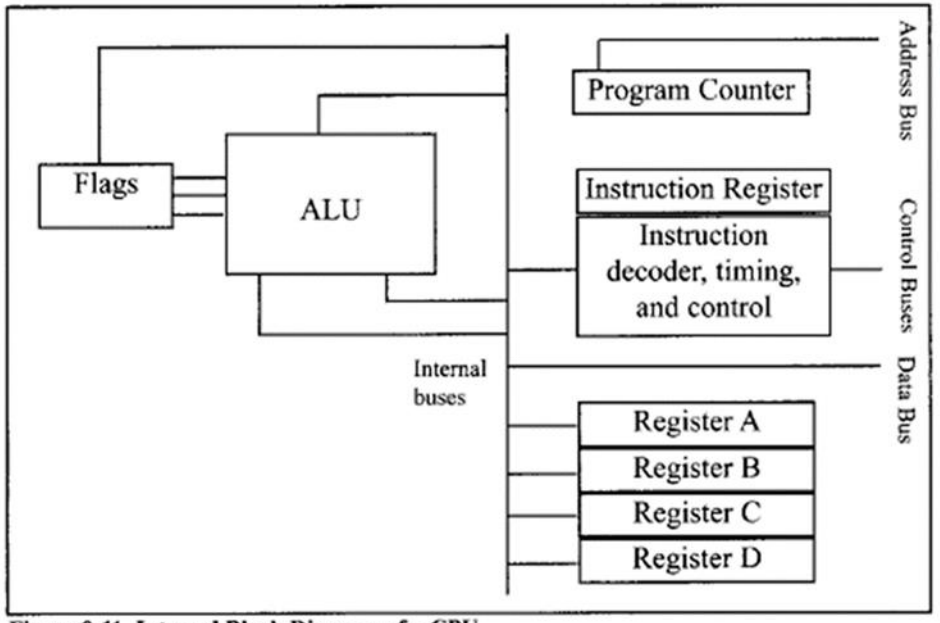


Figure 0-11. Internal Block Diagram of a CPU

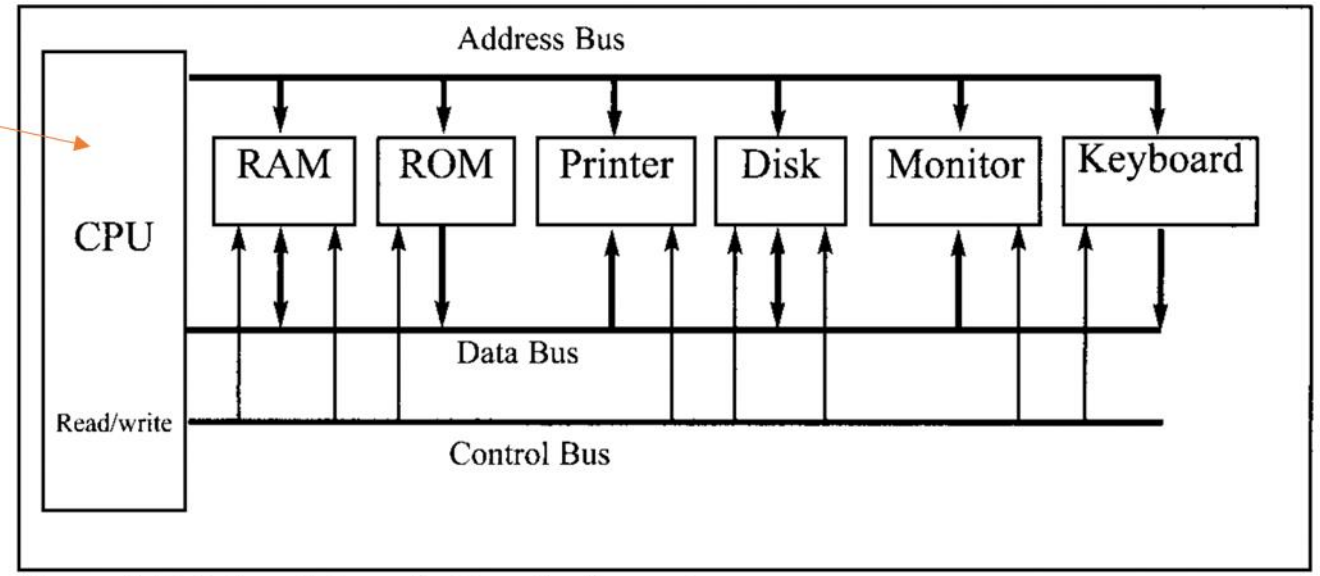


Figure 0-10. Internal Organization of a Computer

CHAPTER 1

THE PIC MICROCONTROLLERS: HISTORY AND FEATURES

SECTION 1.1: MICROCONTROLLERS AND EMBEDDED PROCESSORS

In this section we discuss the need for microcontrollers and contrast them with general-purpose microprocessors such as the Pentium and other x86 microprocessors. We also look at the role of microcontrollers in the embedded market. In addition, we provide some criteria on how to choose a microcontroller.

Microcontroller versus general-purpose microprocessor

What is the difference between a microprocessor and microcontroller? By microprocessor is meant the general-purpose microprocessors such as Intel's x86 family (8086, 80286, 80386, 80486, and the Pentium) or Motorola's PowerPC family. These microprocessors contain no RAM, no ROM, and no I/O ports on the chip itself. For this reason, they are commonly referred to as *general-purpose microprocessors*. See Figure 1-1.

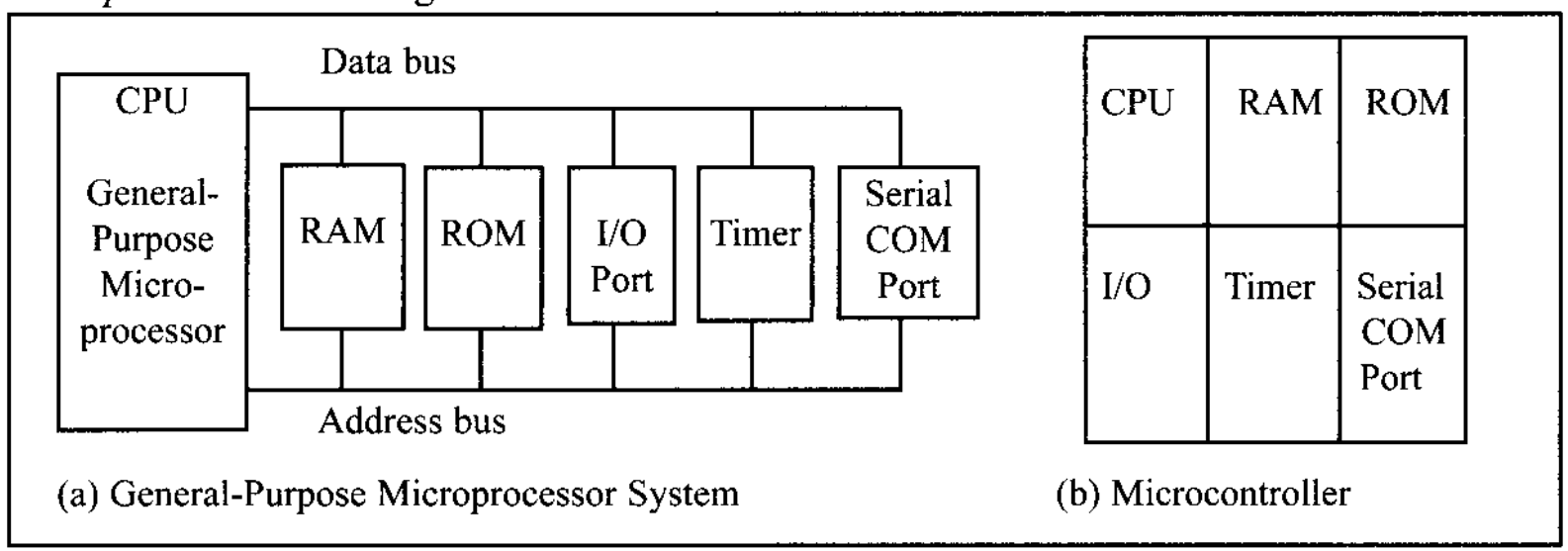


Figure 1-1. Microprocessor System Contrasted With Microcontroller System

Microcontrollers for embedded systems

In the literature discussing microprocessors, we often see the term *embedded system*. Microprocessors and microcontrollers are widely used in embedded system products. An embedded product is controlled by its own internal microprocessor (or microcontroller) as opposed to an external controller. Typically, in an embedded system, the microcontroller's ROM is burned with a purpose for specific functions needed for the system. A printer is an example of an embedded system because the processor inside it performs one task only; namely, getting the data and printing it.

Choosing a microcontroller

There are five major 8-bit microcontrollers. They are: Freescale Semiconductor's (formerly Motorola) 68HC08/68HC11, Intel's 8051, Atmel's AVR, Zilog's Z8, and PIC from Microchip Technology. Each of the above microcontrollers has a unique instruction set and register set; therefore, they are not compatible with each other. Programs written for one will not run on the others. There are also 16-bit and 32-bit microcontrollers made by various chip makers. With all these different microcontrollers, what criteria do designers consider in choosing one? Three criteria in choosing microcontrollers are as follows: (1) meeting the computing needs of the task at hand efficiently and cost effectively; (2) availability of software and hardware development tools such as compilers, assemblers, debuggers, and emulators; and (3) wide availability and reliable sources of the microcontroller. Next, we elaborate on each of the above criteria.

PIC18 features

The PIC18 has a RISC architecture that comes with some standard features such as on-chip program (code) ROM, data RAM, data EEPROM, timers, ADC, and USART and I/O ports. See Figure 1-2. Although the size of the program ROM, data RAM, data EEPROM, and I/O ports varies among the family members, they all have peripherals such as timers, ADC, and USART.

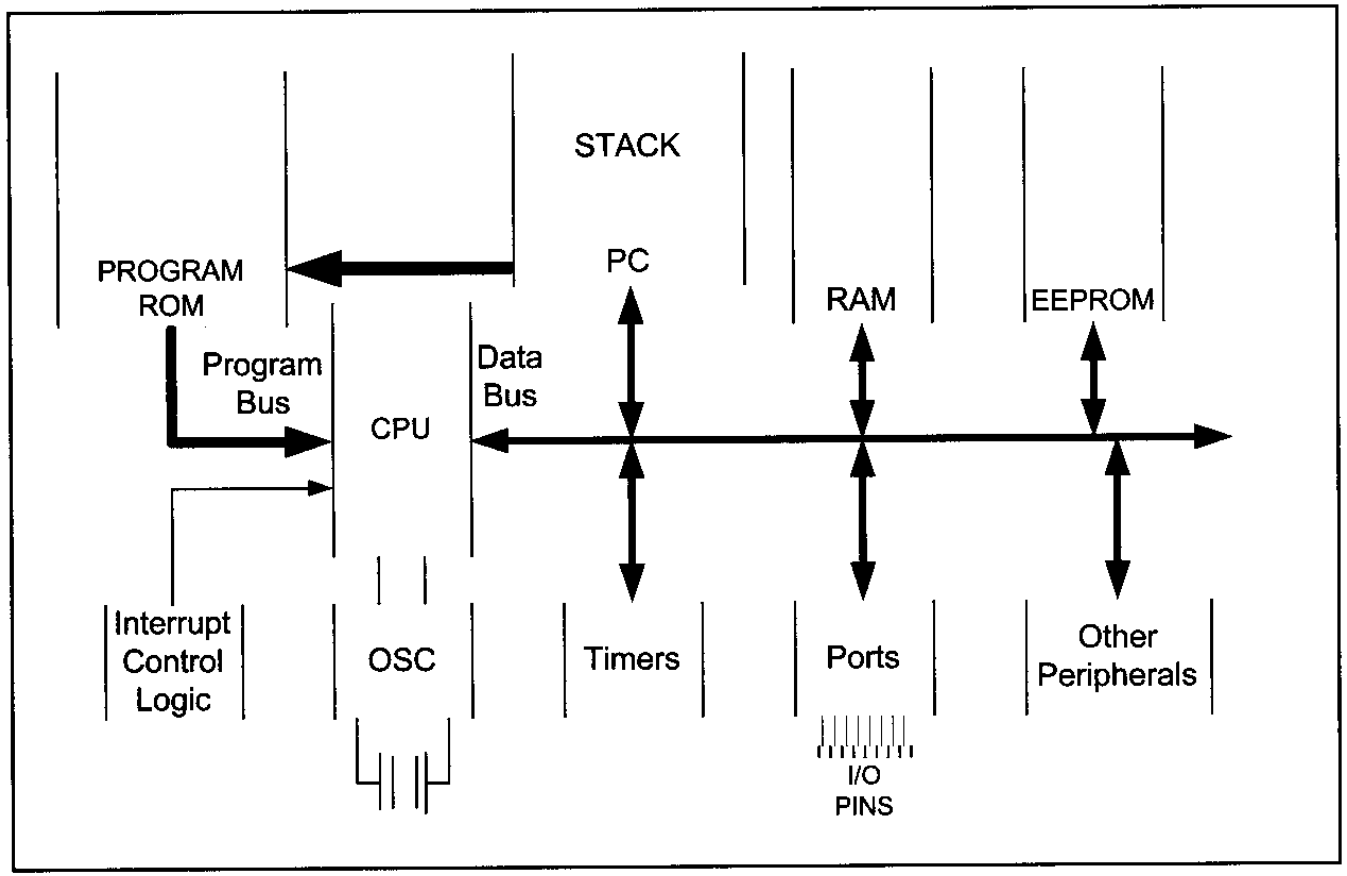


Figure 1-2. Simplified View of a PIC Microcontroller

CHAPTER 2

PIC ARCHITECTURE & ASSEMBLY LANGUAGE PROGRAMMING

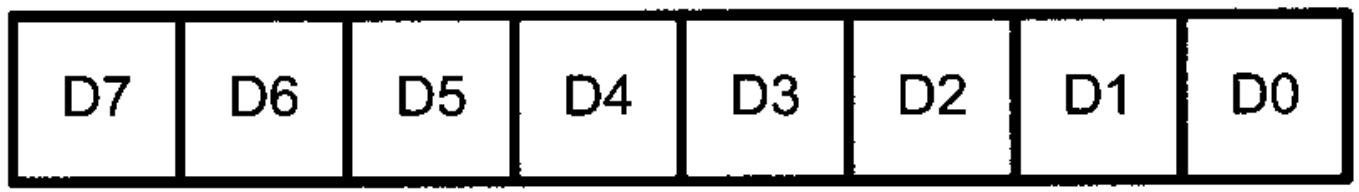
CPUs use many registers to store data temporarily. To program in Assembly language, we must understand the registers and architecture of a given CPU and the role they play in processing data.

SECTION 2.1: THE WREG REGISTER IN THE PIC

PIC microcontrollers have many registers for arithmetic and logic operations. Among them is the WREG register. Because there are a large number of registers inside the PIC, we will concentrate on the widely used register WREG in this section. General-purpose registers are covered in the next section, as well as special function registers. In this section we examine the WREG register of the PIC and show its use with the simple instructions MOVE and ADD.

WREG register

In the CPU, registers are used to store information temporarily. That information could be a byte of data to be processed, or an address pointing to the data to be fetched. The vast majority of PIC registers are 8-bit registers. In the PIC there is only one data type: 8-bit. The 8 bits of a register are shown in the diagram below. These range from the MSB (most-significant bit) D7 to the LSB (least-significant bit) D0. With an 8-bit data type, any data larger than 8 bits must be broken into 8-bit chunks before it is processed.



The 8-bit WREG register is the most widely used register in the PIC microcontroller. WREG stands for working register, as there is only one. The WREG register is the same as the accumulator in other microprocessors. The WREG register is used for all arithmetic and logic instructions. To understand the use of the WREG register, we will show it in the context of two simple instructions: MOVE and ADD.

MOVLW instruction

Simply stated, the MOVLW instruction moves 8-bit data into the WREG register. It has the following format:

```
MOVLW K      ;move literal value K into WREG
```

K is an 8-bit value that can range from 0–255 in decimal, or 00–FF in hex. The L stands for literal, which means, literally, a number must be used. In other words, if we see the word *literal* in any instruction, we are dealing with an actual value that must be provided right there with the instruction. This is similar to the immediate value we see in other microprocessors. Notice that in MOVLW, the letter L (literal) comes first and then the letter W (WREG), which means "move a literal value to WREG," the destination. The following instruction loads the WREG register with a literal value of 25H (i.e., 25 in hex).

```
MOVLW 25H    ;move value 25H into WREG (WREG = 25H)
```

The following instruction loads the WREG register with value 87H (87 in hex).

```
MOVLW 87H    ;load 87H into WREG (WREG = 87H)
```

The following instruction loads the WREG register with value 15H (15 in hex and 21 in decimal).

```
MOVLW 15H    ;load 15H into WREG (WREG = 15H)
```

ADDLW instruction

The ADDLW instruction has the following format:

```
ADDLW K      ;ADD literal value K to WREG
```

The ADD instruction tells the CPU to add the literal value K to register WREG and put the result back in the WREG register. Notice that in ADDLW, first comes the letter L (literal) and then the letter W (WREG), which means "add a literal value to WREG," the destination. To add two numbers such as 25H and 34H, one can do the following:

```
MOVLW 25H    ;load 25H into WREG
ADDLW 34H    ;add value 34 to W(W = W + 34H)
```

Executing the above lines results in WREG = 59H (25H + 34H = 59H)

The following program will add values 12H, 16H, 31H, and 43H:

```
MOVLW 12H    ;load value 12H into WREG (WREG = 12H)
ADDLW 16H    ;add 16 to WREG (WREG = 28H)
ADDLW 11H    ;add 11 to WREG (WREG = 39H)
ADDLW 43H    ;add 43 to WREG (WREG = 7CH)
```

When programming the WREG register of the PIC microcontroller with a literal value, the following points should be noted:

1. Values can be loaded directly into the WREG register. There is no need for a preceding pound sign or dollar sign to indicate that a value is an immediate value as is the case with some other microcontrollers.
2. If values 0 to F are moved into an 8-bit register such as WREG, the rest of the bits are assumed to be all zeros. For example, in "`MOVLW 5H`" the result will be WREG = 05H; that is, WREG = 00000101 in binary.
3. Moving a value larger than 255 (FF in hex) into the WREG register will truncate the upper byte and cause a warning in the .err file.

```
MOVLW 7F2H  ;ILLEGAL 7F2H > 8 bits (FFH), becomes F2H
MOVLW 456H  ;ILLEGAL 456H > FFH, becomes 56H
MOVLW 60A5H ;ILLEGAL but becomes A5H
```

SECTION 2.2: THE PIC FILE REGISTER

The PIC microcontroller has many other registers in addition to the WREG register. They are called data memory space to distinguish them from program (code) memory space. The data memory space in PIC is a read/write (static RAM) memory. In the PIC microcontroller literature, the data memory is also called the *file register*. In this section, we examine the various locations of file register data RAM in the PIC family and discuss their usage with simple instructions such as ADD and MOVE.

File register (data RAM) space allocation in PIC

The file register is read/write memory used by the CPU for data storage, scratch pad, and registers for internal use and functions. As with WREG, we can perform arithmetic and logic operations on many locations of the file register data RAM. The PIC microcontrollers' file register size ranges from 32 bytes to several thousand bytes depending on the chip. Even within the same family, the size of the file register data RAM varies from chip to chip. Notice that the file register data RAM has a byte-size width, just like WREG. The file register data RAM in PIC is divided into two sections: (a) Special Function Registers (SFR), and (b) General-Purpose Registers (GPR). The general-purpose register section is also referred to as General-Purpose RAM (GP RAM). We examine each section separately.

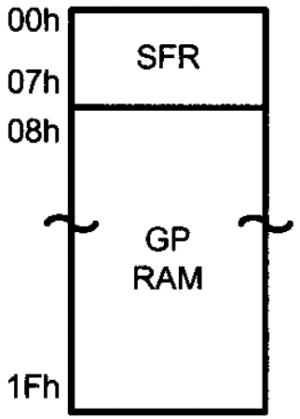
Data memory space/ R/W memory/ Static memory/ File register (Used by CPU for internal storage)

Program (code) memory space

Special Function Register (SFR)

General Purpose Registers (GPR)

PIC12F508



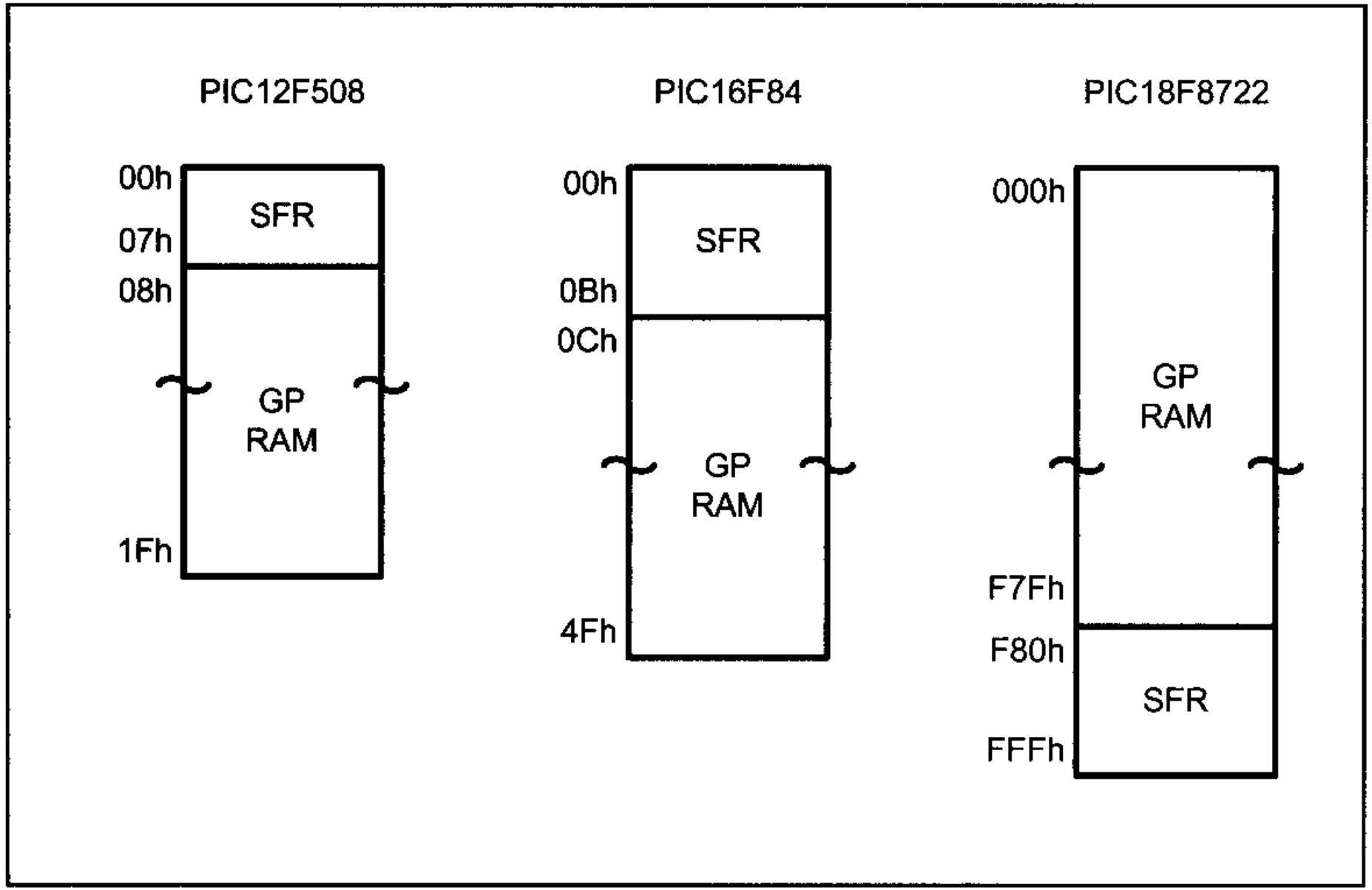
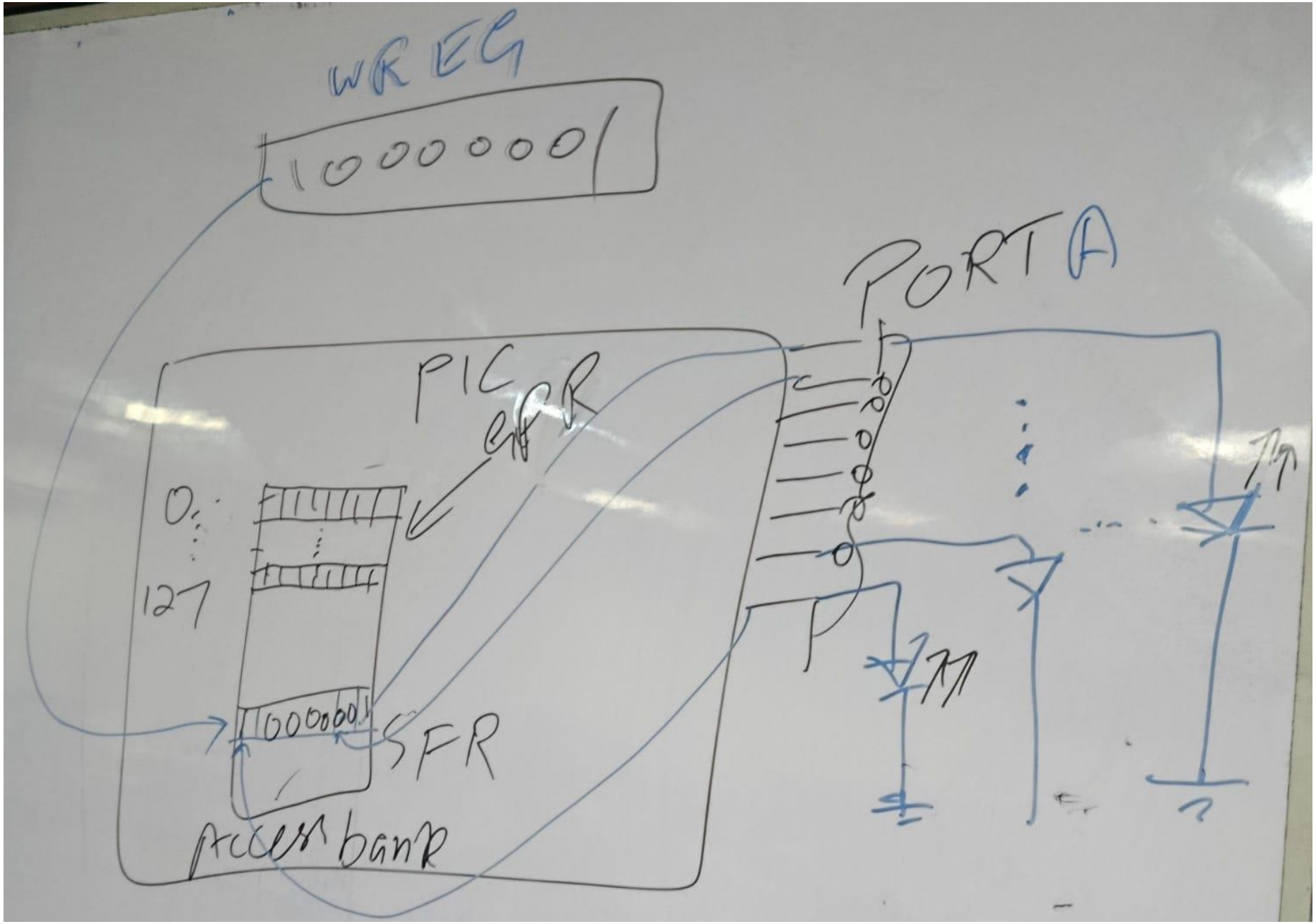


Figure 2-2. File Registers of PIC12, PIC16, and PIC18

SFRs (Special Function Registers)

The Special Function Registers (SFRs) are dedicated to specific functions such as ALU status, timers, serial communication, I/O ports, ADC, and so on. The function of each SFR is fixed by the CPU designer at the time of design because it is used for control of the microcontroller or peripheral. The PIC SFRs are 8-bit registers. The number of locations in the file register set aside for SFR depends on the pin numbers and peripheral functions supported by that chip. That number can vary from chip to chip even among members of the same family.


WREG
10000000
PORTA
PIC
SFR
0
127
SFR
Access bank

File register and access bank in the PIC18

The file register of the PIC18 family can have a maximum of 4096 (4K) bytes. With 4096 bytes, the file register has addresses of 000–FFFH. The file register in the PIC18 is divided into 256-byte banks. Therefore, we can have up to a maximum of 16 banks ($16 \times 256 = 4096$). Although not all members of the PIC18 family have that many banks, every PIC18 family member has at least one bank for the file register. This bank is called the *access bank* and is the default bank when we power up the PIC18 chip. To simplify the discussion of how to use the file register in the PIC family, we focus on this single bank that is found in every member of the PIC18 family.

Examine the access bank for the PIC18 in Figure 2-3. The 256-byte access bank is divided into two equal sections of 128 bytes. These 128-byte sections are given to the general-purpose registers and special function registers. The 128 bytes from locations 00H to 7FH are set aside for general-purpose registers and are used for read/write storage, or what is normally called a *scratch pad*. These 128 locations of RAM are widely used for storing data and parameters by PIC18 programmers and C compilers. Each location of this 128-byte RAM of general-purpose registers can be accessed directly by its address. We will use these locations in future chapters to store data brought into the CPU via I/O and serial ports. We will also use them to define counters for time delay in Chapter 3. The other 128 bytes of the access bank is used for SFRs. It has addresses of F80H to FFFH, as shown

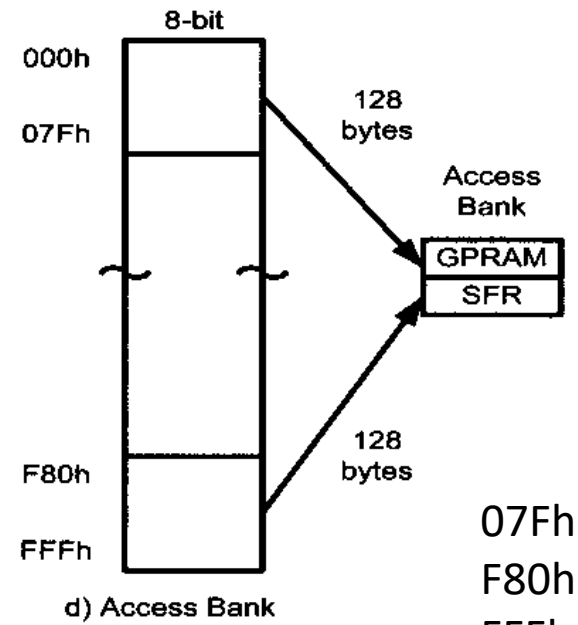


Figure 2-3. File Register for PIC18 Family

07Fh = 127
F80h = 3968
FFFh = 4095

SECTION 2.3: USING INSTRUCTIONS WITH THE DEFAULT ACCESS BANK

The instructions we have used so far are the literal (constant) value of K and the WREG register. They also used the WREG register as their destination. We saw simple examples of using MOVLW and ADDLW earlier in Section 2.1. The PIC allows direct access to other locations in the file register for ALU and other operations. In this section we show the instructions using various locations of the file register. This is one of the most important sections in the book for mastering the topic of PIC Assembly language programming.

MOVWF instruction

As we discussed in the last section, the access bank of the file register is the default bank upon powering up the PIC18. The term *file register* must be emphasized because the instructions have the letter F in their mnemonics. In instructions such as MOVWF, the F stands for a location in the file register, while W means WREG. The MOVWF instruction tells the CPU to move (in reality, copy) the source register of WREG to a destination in the file register (F). After this instruction is executed, the location in the file register will have the same value as register WREG. The location in the file register can be one of the SFRs or a location in the general purpose registers region. For example, the "MOVWF PORTA" instruction will move the contents of WREG into the SFR register called PORTA. The following program first loads the WREG register with value 55H, then moves this value around to various SFRs of ports B, C, and D:

```
MOVLW 55H          ;WREG = 55H
MOVWF PORTB        ;copy WREG to Port B (Port B = 55H)
MOVWF PORTC        ;copy WREG to Port C (Port C = 55H)
MOVWF PORTD        ;copy WREG to Port D (Port D = 55H)
```

PORTB, PORTC, and PORTD are part of the special function registers in the file register, as was shown in Figure 2-4. They can be connected to the I/O pins of the PIC microcontroller as we will see in Chapter 4. We can also move (copy) the contents of WREG into any location in the general-purpose registers (RAM) region of the file registers. The following program will put 99H into locations 0–4 of the GPR region in the file register:

```
MOVLW 99H          ;WREG = 99H
MOVWF 0H           ;move (copy) WREG contents to location 0h
MOVWF 1H           ;move (copy) WREG contents to location 1h
MOVWF 2H
MOVWF 3H
MOVWF 4H
```

The chart indicates the contents of addresses 0–4 after execution of the code.

Address	Data
000	99
001	99
002	99
003	99
004	99

Example 2-1

State the contents of file register RAM locations after the following program:

```
MOVLW 99H          ;load WREG with value 99H
MOVWF 12H
MOVLW 85H          ;load WREG with value 85H
MOVWF 13H
MOVLW 3FH          ;load WREG with value 3FH
MOVWF 14H
MOVLW 63H          ;load WREG with value 63H
MOVWF 15H
MOVLW 12H          ;load WREG with value 12H
MOVWF 16H
```

Solution:

After the execution of `MOVWF 12H` fileReg RAM location 12H has value 99H;
After the execution of `MOVWF 13H` fileReg RAM location 13H has value 85H;
After the execution of `MOVWF 14H` fileReg RAM location 14H has value 3FH;
After the execution of `MOVWF 15H` fileReg RAM location 15H has value 63H;
And so on, as shown in the chart.

Address	Data
012	99
013	85
014	3F
015	63
016	12

More instructions involving the WREG and the access bank

There is a group of logic and arithmetic instructions that involve both the WREG and a location in the file register. The ADDWF instruction is one of them. The ADDWF instruction adds together the contents of WREG and a file register location. The file register location can be one of the SFRs or a general-purpose register. The destination for the result can be the WREG or the file register. The following format indicates the destination:

```
ADDWF fileReg, D
```

where fileReg is the file register location and D indicates the destination bit. The D bit can be 0 or 1. If D = 0, it means that the destination is WREG. If D = 1, then the result will be placed in the file register.

The following will first put value 22H into GP RAM locations 5, 6, and 7, then add them together and put the result in WREG:

```
MOVLW 22H    ;WREG = 22H
MOVWF 5H     ;move(copy) WREG contents to location 5H
MOVWF 6H     ;move(copy) WREG contents to location 6H
MOVWF 7H     ;move(copy) WREG contents to location 7H
ADDWF 5H, 0 ;add W and loc 5, result in WREG (W = 44H)
ADDWF 6H, 0 ;add W and loc 6, result in WREG (W = 66H)
ADDWF 7H, 0 ;add W and loc 7, result in WREG (W = 88H)
```

Address	Data
005	22
006	22
007	22

GPR after the execution up to "MOVWF 7H"
WREG = 22H

Address	Data
005	22
006	22
007	22

GPR after the execution up to "ADDWF 7H, 0"
WREG = 88H

What would be the output of the following program?

```
MOVLW 22H
MOVWF 5H
MOVWF 6H
MOVWF 7H
ADDWF 5, 0
ADDWF 6, 0
ADDWF 7, 1
```

```
MOVLW 22H    ;WREG = 22H
MOVWF 5H     ;move (copy) WREG contents to location 5H
MOVWF 6H     ;move (copy) WREG contents to location 6H
MOVWF 7H     ;move (copy) WREG contents to location 7H
ADDWF 5, 0   ;add W and loc 5, result in WREG (W = 44H)
ADDWF 6, 0   ;add W and loc 6, result in WREG (W = 66H)
ADDWF 7, 1   ;add W and loc 7, result in location 7H
             ;now location 7 has 88H and WREG = 66H
```

Address	Data
005	22
006	22
007	22

GP RAM after the execution up to "MOVWF 7H"
WREG = 22H

Address	Data
005	22
006	22
007	88

GP RAM after the execution up to "ADDWF 7H, 1"
WREG = 66H

To make things less confusing as far as the D bit is concerned, the PIC assembler allows us to use the letters W or F instead of 0 or 1 to indicate the destination. Look at the following two formats:

```
ADDWF fileReg, w  ;add WREG and fileReg. WREG = the result
ADDWF fileReg, f  ;add WREG and fileReg
                  ;fileReg = the result
```

What would be the output of the following program?

```
MOVLW 22H
MOVWF 5H
MOVWF 6H
MOVWF 7H
ADDWF 5H,W
ADDWF 6H,W
ADDWF 7H,F
```

Example 2-2

State the contents of file register RAM locations 12H and WREG after the following program:

```
MOVLW 0          ;move 0 WREG to clear it (WREG = 0)
MOVWF 12H        ;move WREG to location 12 to clear it
MOVLW 22H        ;load WREG with value 22H
ADDWF 12H, F     ;add WREG to loc 12H, loc 12 = sum
ADDWF 12H, F     ;add WREG to loc 12H, loc 12 = sum
ADDWF 12H, F     ;add WREG to loc 12H, loc 12 = sum
ADDWF 12H, F     ;add WREG to loc 12H, loc 12 = sum
```

Solution:

Example 2-2

State the contents of file register RAM locations 12H and WREG after the following program:

```
MOVLW 0          ;move 0 WREG to clear it (WREG = 0)
MOVWF 12H        ;move WREG to location 12 to clear it
MOVLW 22H        ;load WREG with value 22H
ADDWF 12H, F     ;add WREG to loc 12H, loc 12 = sum
ADDWF 12H, F     ;add WREG to loc 12H, loc 12 = sum
ADDWF 12H, F     ;add WREG to loc 12H, loc 12 = sum
ADDWF 12H, F     ;add WREG to loc 12H, loc 12 = sum
```

Solution:

The program clears both the WREG and RAM location 12H in the file register. Then it loads WREG with value 22H. From then on, it adds the WREG register and location 12 together and saves the result in location 12H. It does that four times. At the end, location 12H of GP RAM has the value of 88H ($4 \times 22H = 88H$) and WREG = 22H.

After each "`ADDWF 12, F`" instruction

Address	Data
011	
012	22
013	

`WREG = 22H`

Address	Data
011	
012	44
013	

`WREG = 22H`

Address	Data
011	
012	66
013	

`WREG = 22H`

Address	Data
011	
012	88
013	

`WREG = 22H`

Example 2-3

Rewrite the last example to place the sum in WREG as you add the file register locations and the WREG register.

```
MOVLW 0          ;move 0 WREG to clear it (WREG = 0)
MOVWF 12H        ;move WREG to location 12 to clear it
MOVLW 22H        ;load WREG with value 22H
ADDWF 12H, W     ;add WREG and loc 12H, WREG = sum
ADDWF 12H, W     ;add WREG and loc 12H, WREG = sum
ADDWF 12H, W     ;add WREG and loc 12H, WREG = sum
ADDWF 12H, W     ;add WREG and loc 12H, WREG = sum
```

Example 2-3

Rewrite the last example to place the sum in WREG as you add the file register locations and the WREG register.

```
MOVLW 0          ;move 0 WREG to clear it (WREG = 0)
MOVWF 12H        ;move WREG to location 12 to clear it
MOVLW 22H        ;load WREG with value 22H
ADDWF 12H, W     ;add WREG and loc 12H, WREG = sum
ADDWF 12H, W     ;add WREG and loc 12H, WREG = sum
ADDWF 12H, W     ;add WREG and loc 12H, WREG = sum
ADDWF 12H, W     ;add WREG and loc 12H, WREG = sum
```

Solution:

The program adds WREG and location 12H together and saves the result in WREG each time. At the end, location 12H has a value of 22H and WREG = 88H ($4 \times 22H = 88H$).

After each "`ADDWF 12, W`" instruction:

Address	Data
011	
012	22
013	

`WREG = 22H`

Address	Data
011	
012	22
013	

`WREG = 44H`

Address	Data
011	
012	22
013	

`WREG = 66H`

Address	Data
011	
012	22
013	

`WREG = 88H`